Architecture Big Data appliquée à l'IA

Séance 4 : streaming et Kafka

Séance 4 : streaming et Kafka

Support de cours, fil conducteur ShopFlow.
Objectif : comprendre le modèle du **streaming**, les **garanties de livraison** et le **temps des événements**, pour construire au TP un pipeline temps réel de bout en bout.

1. Batch, micro-batch, streaming

1.1 Trois manières de traiter

- **Batch** : on traite un ensemble **borné** de données (les paiements d'hier). Résultat complet, mais latence de plusieurs heures. C'est ce que nous avons fait en séances 1 à 3.
- **Streaming** : on traite un flux **non borné**, événement par événement, au fil de l'eau. Latence de quelques millisecondes à quelques secondes.
- **Micro-batch** : on découpe le flux en petits lots de quelques secondes et on applique un traitement batch à chacun. C'est le modèle historique de Spark Structured Streaming.

1.1 Trois manières de traiter (suite)

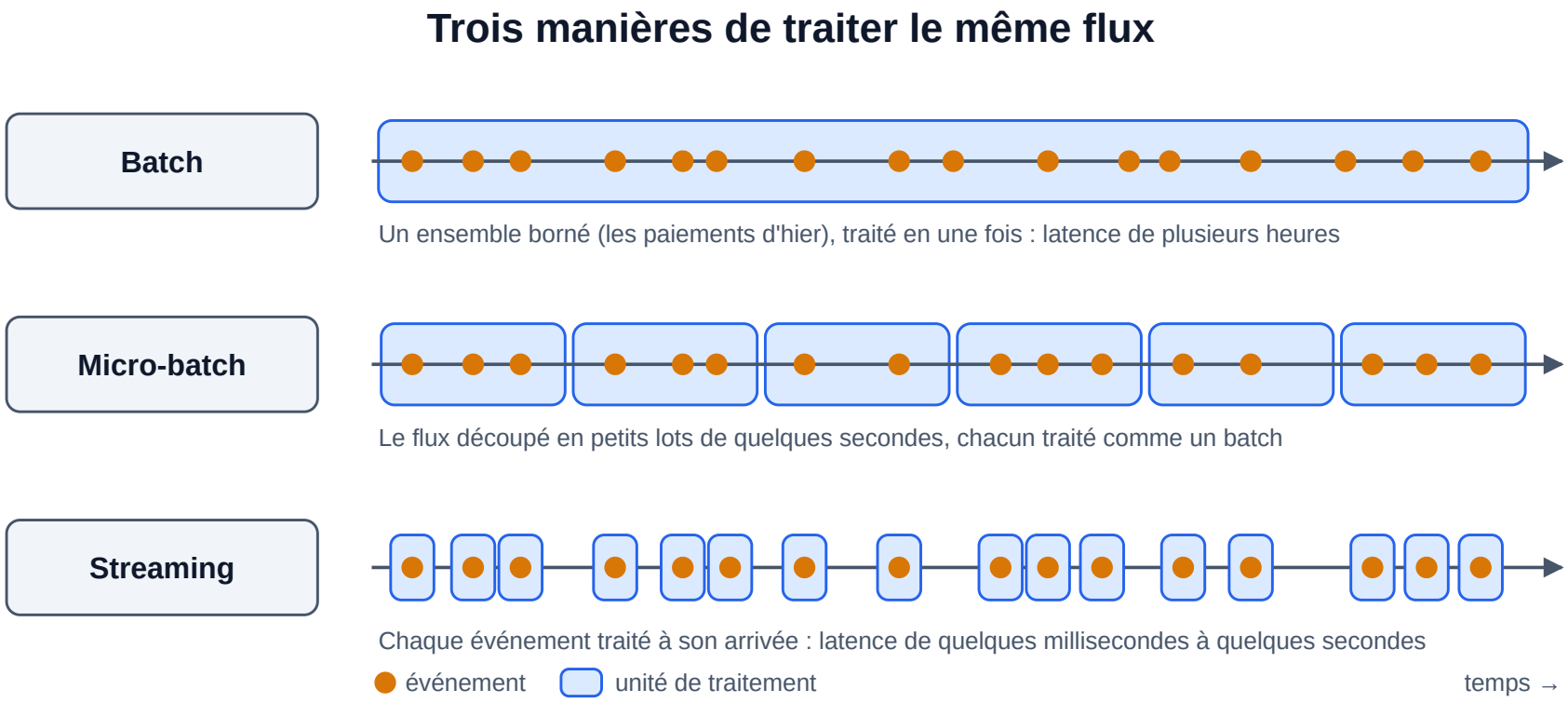


Schéma : les mêmes événements (points orange) sont traités en un seul lot borné, en petits lots successifs, ou un par un à leur arrivée ; la latence diminue de haut en bas.

1.2 Exemple dans le contexte du TP

Besoin	Latence acceptable	Mode
Bloquer un paiement frauduleux	< 100 ms, pendant l'autorisation	Scoring en ligne (séance 5)
Détecter un test de cartes en cours (rafale sur une IP)	Quelques secondes à une minute	Streaming
Tableau de bord des ventes du jour	Une minute	Micro-batch
Rapport mensuel du chiffre d'affaires	Un jour	Batch

Latence ou débit : un traitement batch amortit les coûts fixes (lecture, planification) sur beaucoup de données ; un traitement continu paie ces coûts en permanence. Le streaming coûte plus cher par événement : on ne le choisit que si la fraîcheur a de la valeur.

2. Le journal comme abstraction

2.1 Un journal, c'est quoi

Un **journal** (*log*) est une suite d'enregistrements **ajoutés à la fin uniquement**, ordonnés, chacun identifié par sa position (l'**offset**). On ne modifie jamais un enregistrement ; on lit à partir d'une position.

Nous en avons déjà vu un :

- le `_delta_log` de Delta Lake (séance 2), qui ordonne les transactions d'une table.

2.2 L'idée fondatrice

Si toutes les applications publient leurs changements dans un journal partagé, chaque système qui a besoin de ces données (un **consommateur**) **lit le journal à son propre rythme** et en tire sa propre version des données. Le schéma ci-dessous montre trois consommateurs du **même** journal de paiements (exemple du TP). Ils lisent exactement les mêmes messages, mais chacun en fait autre chose.

Prenons un seul message : « *paiement n° 42, carte 1234, 89 €, 14 h 03* ».

Consommateur	Ce qu'il fait du message	À quoi il sert	Retard acceptable
Index (moteur de recherche, par exemple Elasticsearch)	Range le paiement dans un « annuaire » où l'on retrouve vite un paiement par numéro, e-mail ou nom de client	Le service client tape « paiement 42 » ou l'e-mail du client et obtient la réponse en une fraction de seconde	Quelques minutes : personne n'est lésé
Fraude (séance 5)	Met à jour des features d'une carte et donne un score de risque	Bloquer un paiement suspect **avant** l'envoi de la commande	Quelques secondes au plus
Entrepôt (data warehouse, séance 1)	Ajoute une ligne à la table historique des paiements, chargée par lots	Rapports et tableaux de bord : chiffre d'affaires du mois, taux de fraude par pays	Une heure, voire une journée

2.2 L'idée fondatrice (suite)

Pourquoi trois lecteurs **indépendants** plutôt qu'un seul programme qui ferait tout ?

- **Besoins différents** : la fraude doit être rapide, l'entrepôt préfère de gros lots, l'index a son propre format. Un programme unique serait aussi lent que la plus lente des trois tâches.
- **Pannes isolées** : si l'index tombe en panne (sur le schéma, il est en retard à l'offset 2), la fraude continue de bloquer les paiements suspects.
- **Équipes différentes** : chaque équipe fait évoluer son consommateur sans toucher aux autres ni au site qui produit les paiements.

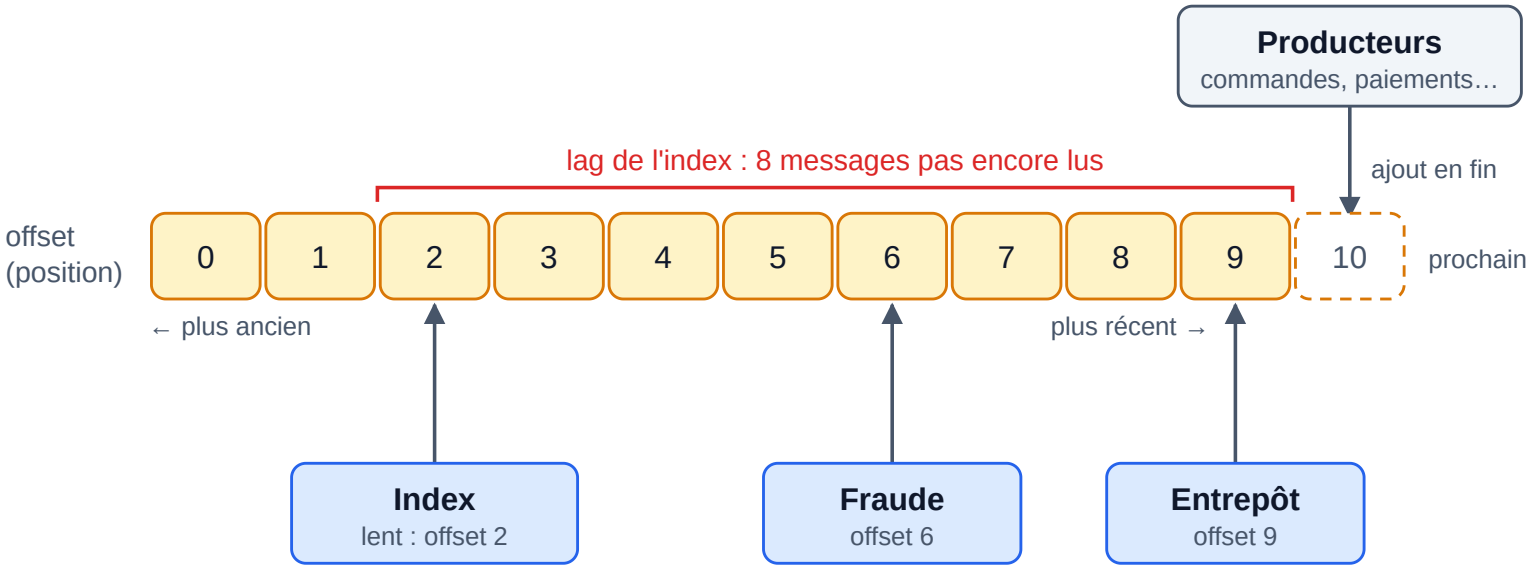
2.2 L'idée fondatrice (suite)

Sans journal, le site devrait envoyer lui-même chaque paiement à l'index, au détecteur de fraude et à l'entrepôt, et attendre que les trois répondent. Avec le journal, il écrit une seule fois. On dit que le journal **découple** producteurs et consommateurs :

- un consommateur lent ou en panne ne bloque personne ; il reprend à son offset (comment cet offset est défini : voir 3.4) ;
- ajouter un consommateur ne demande aucune modification des producteurs ;
- on peut **rejouer** l'historique pour reconstruire un état ou tester un nouveau traitement.

2.2 L'idée fondatrice (suite)

Un journal partagé, lu à des rythmes différents



Chaque consommateur garde sa position ; on ne modifie jamais un enregistrement.
Rejouer = relire depuis un offset plus ancien (par exemple 0).

Schéma : les producteurs ajoutent toujours en fin de journal. Les trois consommateurs (index de recherche, détection de fraude, entrepôt) lisent les mêmes messages, chacun à sa propre position : l'index, en retard, ne gêne ni les producteurs ni les autres lecteurs.

2.2 L'idée fondatrice (suite)

Pourquoi l'offset rend la lecture rapide. L'offset est un simple numéro de position, comme un marque-page dans un livre. Un consommateur dit seulement au serveur Kafka : « donne-moi les messages à partir de l'offset 6 ».

- **Aller directement au bon endroit** : Kafka tient un petit index « offset → position dans le fichier ». Il saute donc tout de suite au message 6, sans parcourir ni chercher parmi les messages déjà lus.
- **Lire à la suite** : à partir de là, il suffit de lire les messages dans l'ordre où ils sont rangés sur le disque. C'est la lecture séquentielle, la plus rapide (séance 1). Les messages récents sont en général encore en RAM (voir 3.2).
- **Un seul nombre à retenir** : une file classique note, message par message, ce qui a été lu et acquitté. Kafka, lui, ne retient qu'un nombre par consommateur et par partition. Rien à mettre à jour pour chaque message lu, et ajouter un lecteur ne coûte presque rien.

2.2 L'idée fondatrice (suite)

- **Mesurer le retard** : offset du dernier message écrit moins offset du consommateur = le **lag** (8 messages pour l'index sur le schéma). Si le lag grandit, ce consommateur lit moins vite que les producteurs n'écrivent.

Kafka est né de cette idée chez LinkedIn, publié en 2011, puis confié à la fondation Apache.

2.3 Journal et file de messages

	File de messages classique	Journal (Kafka)
Après lecture	Le message est supprimé (acquitté)	Le message reste jusqu'à expiration de la rétention
Plusieurs lecteurs indépendants	Il faut dupliquer la file	Chacun garde son offset
Relecture	Impossible	Possible depuis n'importe quel offset conservé
Ordre	Souvent non garanti avec plusieurs consommateurs	Garanti dans une partition

Kafka 4.0 ajoute en *early access* des « share groups » qui donnent une sémantique de file au-dessus du journal (KIP-932) ; nous ne les utiliserons pas.

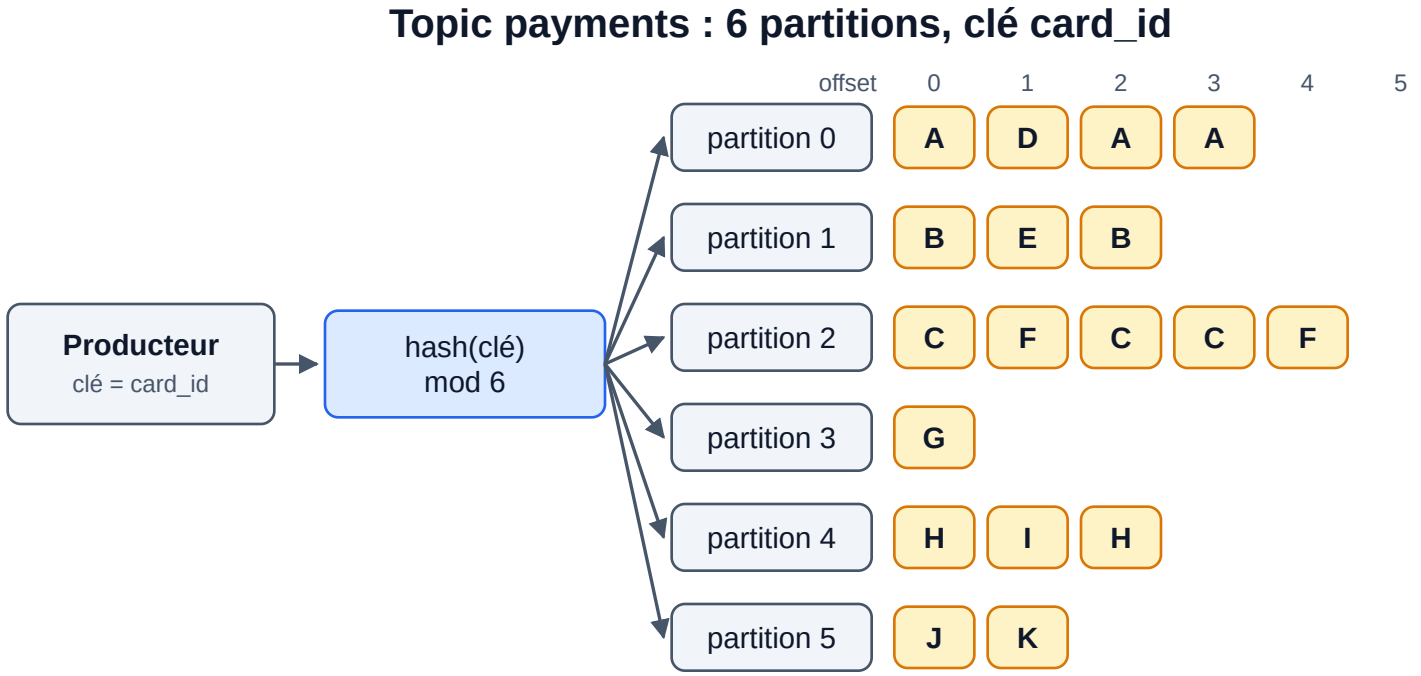
3. Kafka en détail

3.1 Topics, partitions, offsets

- Un **topic** est un flux nommé (`payments`, `clicks`).
- Il est découpé en **partitions** : chacune est un journal indépendant, stocké sur un broker. Les partitions permettent le parallélisme : plusieurs brokers écrivent, plusieurs consommateurs lisent en même temps.
- Un enregistrement a une **clé**, une **valeur**, un horodatage et des en-têtes. Le producteur choisit la partition en hachant la clé : **tous les enregistrements d'une même clé vont dans la même partition**, donc sont lus dans l'ordre.
- **L'ordre n'est garanti que dans une partition**, jamais à l'échelle du topic.

Pour ShopFlow, la clé de `payments` est le `card_id` : tous les paiements d'une carte arrivent dans l'ordre au même consommateur, qui peut tenir l'état de la carte (nombre de transactions de la dernière heure) sans coordination. C'est ce que fera le scoreur de la séance 5.

3.1 Topics, partitions, offsets (suite)



A, B, C… = valeurs de card_id. Même clé → même partition → lue dans l'ordre.
Aucun ordre garanti entre deux partitions différentes.

Schéma : toutes les occurrences d'une même carte (A, C…) tombent dans la même partition, où elles sont ordonnées par offset ; chaque partition a ses propres offsets.

3.2 Pourquoi Kafka est rapide (lien avec la séance 1)

- **Écriture séquentielle** en fin de fichier : c'est le motif d'accès le plus efficace sur disque comme sur SSD.
- **Zéro copie** : pour envoyer des données à un consommateur, le noyau copie directement du cache de pages vers la carte réseau, sans passer par la mémoire de l'application.
- **Lots et compression** : producteurs et consommateurs échangent des lots d'enregistrements compressés.

3.3 Réplication et durabilité

- Chaque partition a un **facteur de réplication** (souvent 3) : un **leader**, qui reçoit écritures et lectures, et des **followers** qui copient son journal.
- Les répliques à jour forment l'**ISR** (*in-sync replicas*). Si le leader tombe, un membre de l'ISR prend le relais.
- Côté producteur, `acks` fixe quand une écriture est confirmée :
  - `acks=0` : sans attendre ; perte possible ;
  - `acks=1` : quand le leader a écrit ; perte si le leader tombe avant réplication ;
  - `acks=all` : quand toutes les répliques de l'ISR ont écrit. Avec `min.insync.replicas=2` sur un facteur 3, on tolère la perte d'un broker sans perte de données.

3.3 Réplication et durabilité (suite)

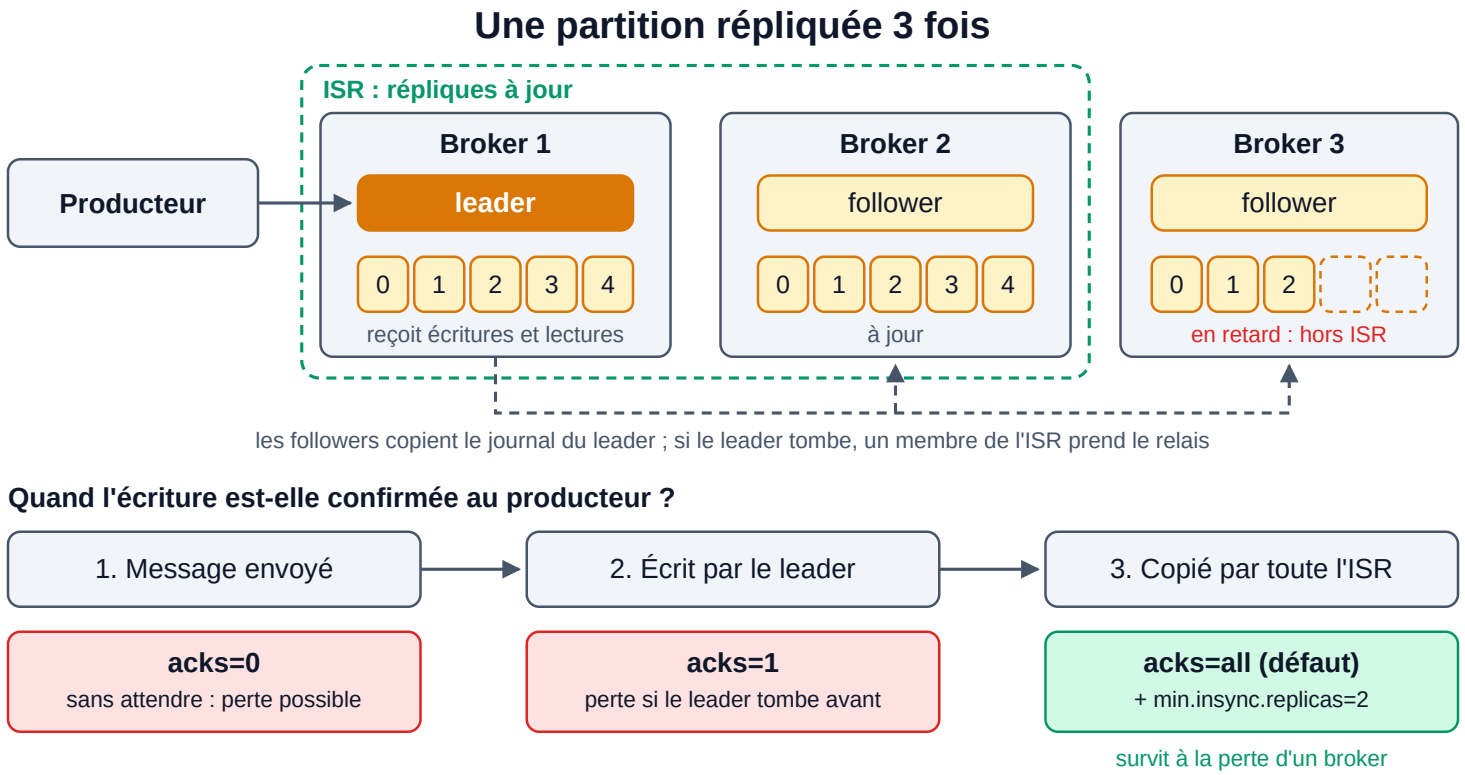


Schéma : le producteur écrit sur le leader, les followers copient son journal ; le réglage `acks` choisit à quelle étape l'écriture est confirmée.

3.4 Consommateurs et groupes

- Un consommateur lit une partition à partir d'un offset et **enregistre (commit)** l'offset atteint dans un topic interne (`__consumer_offsets`).

3.4 Consommateurs et groupes (suite)

Comment l'offset d'un consommateur est défini. L'offset n'appartient pas au consommateur seul mais à son **groupe** (`group.id`, voir plus bas) : Kafka garde un offset par groupe et par partition, et cet offset désigne le **prochain message à lire**. Il est fixé à trois moments :

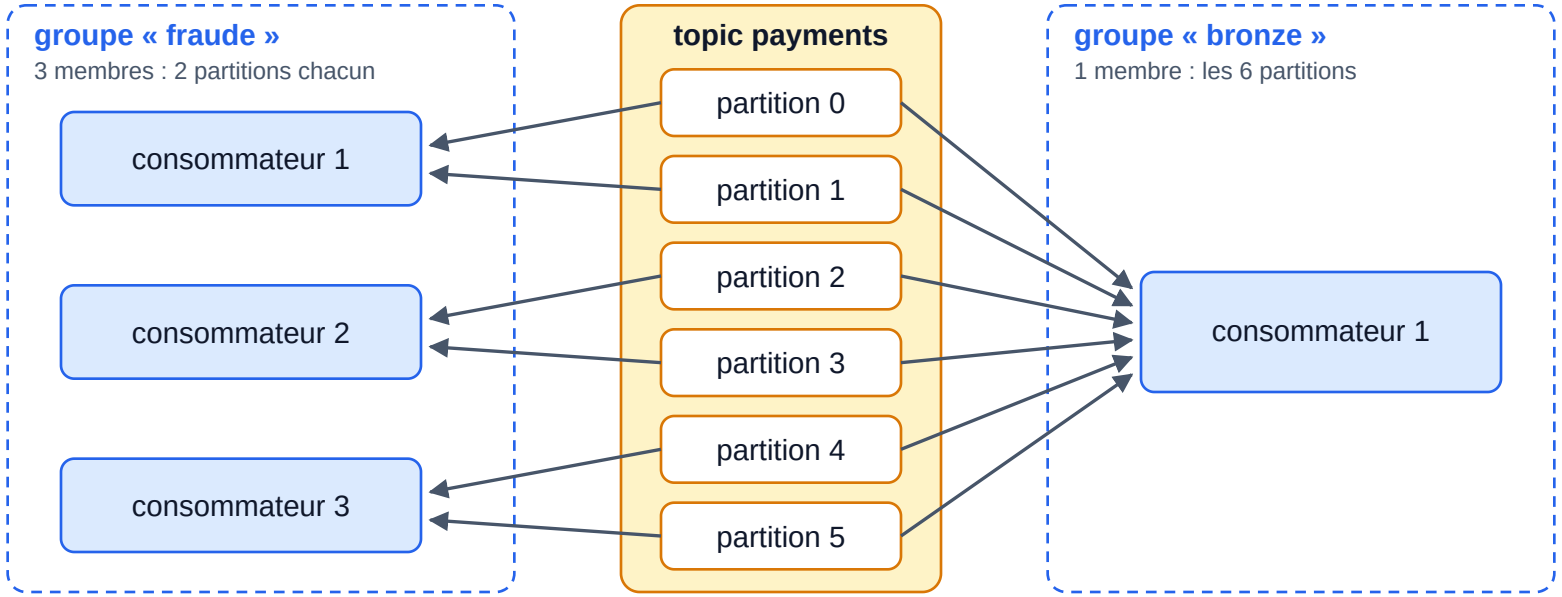
1. **Au démarrage** : le consommateur reprend à l'offset déjà enregistré pour son groupe. Pour un groupe tout neuf, il n'y en a pas encore ; le réglage `auto.offset.reset` décide alors : `latest` (par défaut) = ne lire que les nouveaux messages, `earliest` = partir du plus ancien message encore conservé.
2. **Pendant la lecture** : le consommateur avance dans sa mémoire, puis **commite** sa position, soit automatiquement toutes les 5 secondes (`enable.auto.commit=true`, par défaut), soit à la main après avoir traité les messages. Le moment de ce commit détermine les garanties de livraison (3.5).
3. **À la demande** : on peut déplacer l'offset pour rejouer l'historique, dans le code (`seek()`) ou avec l'outil `kafka-consumer-groups.sh --reset-offsets`.

3.4 Consommateurs et groupes (suite)

- Un **groupe de consommateurs** (`group.id`) se partage les partitions d'un topic : **chaque partition est lue par un seul membre du groupe**. Avec 6 partitions, au-delà de 6 consommateurs, les suivants restent inactifs.
- Quand un membre arrive ou part, le groupe se **rééquilibre** (*rebalance*) : les partitions sont réattribuées. Kafka 4.0 rend disponible le nouveau protocole de groupe (KIP-848), où le broker pilote l'attribution de manière incrémentale, ce qui évite d'arrêter tout le groupe à chaque changement.
- Deux groupes différents lisent **le même flux indépendamment** : au TP, chaque binôme a son groupe.
- Le **lag** d'un consommateur sur une partition = offset de fin de la partition (prochain offset à écrire) − offset commité. Un lag qui croît signifie que le consommateur ne suit pas.

3.4 Consommateurs et groupes (suite)

Deux groupes lisent le même topic



Dans un groupe, chaque partition est lue par un seul membre (au-delà de 6 membres : inactifs).
Chaque groupe a ses propres offsets commités : il lit tout le flux, indépendamment des autres.

Schéma : dans le groupe de gauche, les 6 partitions sont partagées entre 3 membres ; le groupe de droite, avec un seul membre, reçoit pourtant tout le flux.

3.5 Garanties de livraison

Tout dépend du **moment où l'on commite l'offset** par rapport au traitement :

Sémantique de livraison de message (Message Delivery Semantics)

Garantie	Mécanisme	Risque
at-most-once (Au plus une fois)	Commit avant traitement	Une panne entre les deux perd les messages
at-least-once (Au moins une fois)	Commit après traitement	Une panne entre les deux fait retraiter : doublons
exactly-once (Exactement une fois)	Transactions Kafka (lecture-traitement-écriture dans Kafka, `isolation.level=read_committed`) : regrouper l'écriture dans plusieurs partitions et la validation des offsets du consommateur dans une opération atomique : soit tout est validé, soit tout est annulé (rollback); ou sortie **idempotente**	Plus complexe, plus coûteux

En pratique, la combinaison la plus courante est **au moins une fois + sortie idempotente** : on écrit de façon qu'un retraitement produise le même résultat (fichier nommé par l'offset de départ, `MERGE` sur une clé, déduplication par identifiant). C'est ce que vous ferez au TP.

3.5 Garanties de livraison (suite)

Le moment du commit fixe la garantie

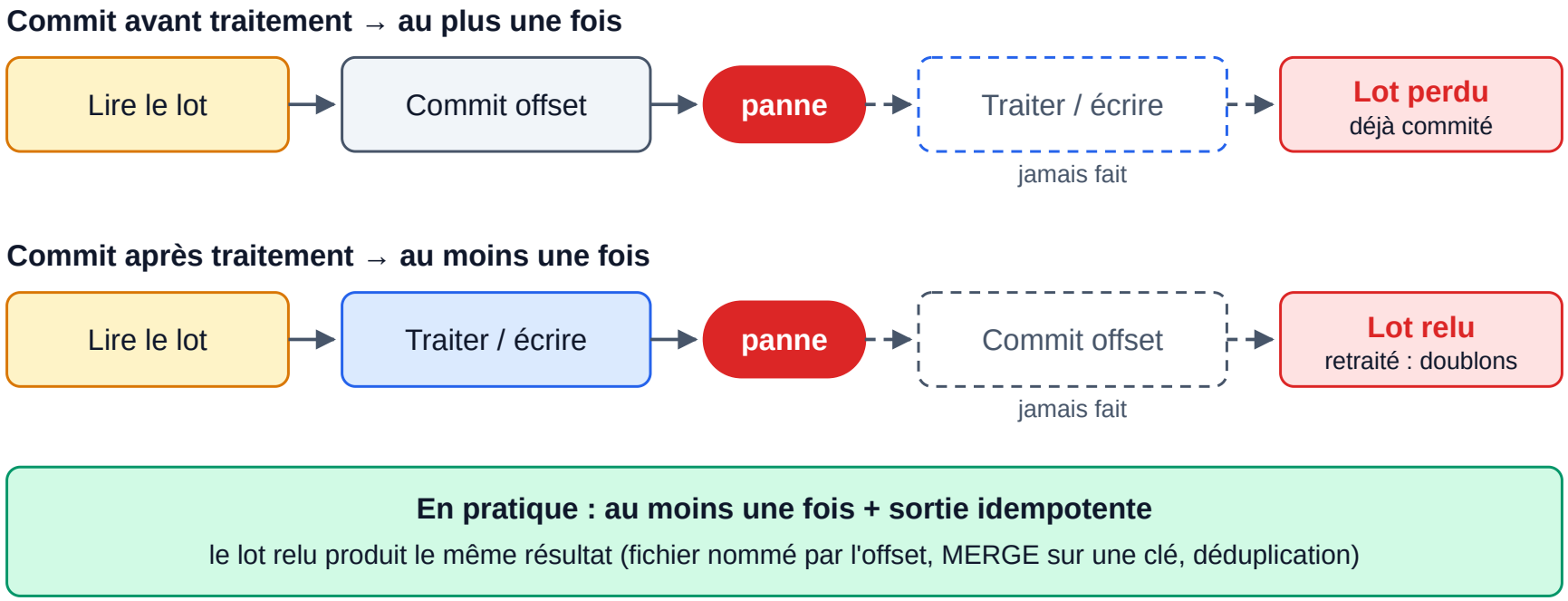


Schéma : une panne au même endroit perd le lot si l'on a commité avant de traiter, et le fait retraiter si l'on commite après ; une sortie idempotente rend ce retraitement sans effet.

3.6 Rétention et compaction

- **Rétention** par durée (`retention.ms`, 7 jours par défaut) ou par taille ; les segments les plus anciens sont supprimés. C'est elle qui borne la possibilité de rejouer.
- **Compaction** (`cleanup.policy=compact`) : Kafka garde au moins le **dernier enregistrement de chaque clé**. Le topic devient une table « clé → dernière valeur » qu'on peut relire en entier (par exemple le dernier profil de chaque client).

3.6 Rétention et compaction (suite)

Rétention et compaction d'un même journal

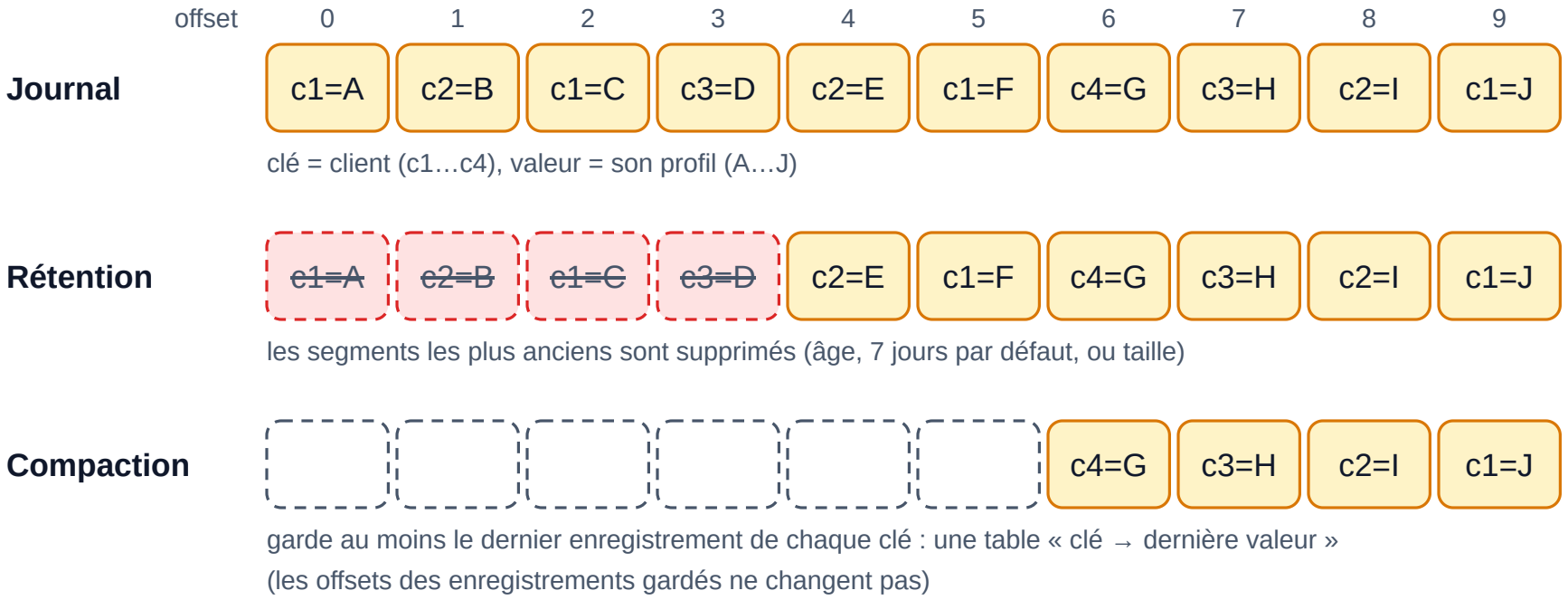


Schéma : la rétention coupe le début du journal quelle que soit la clé ; la compaction efface les anciennes valeurs de chaque clé et ne garde que la plus récente.

3.7 KRaft : la fin de ZooKeeper

Jusqu'en version 3, Kafka stockait ses métadonnées (topics, partitions, leaders) dans un cluster **ZooKeeper** séparé. Depuis **Kafka 4.0 (mars 2025)**, ils sont déplacées dans un quorum de **contrôleurs Kafka**. Conséquence : un seul système à déployer et à surveiller.

3.7 KRaft : la fin de ZooKeeper (suite)

Où vivent les métadonnées du cluster ?

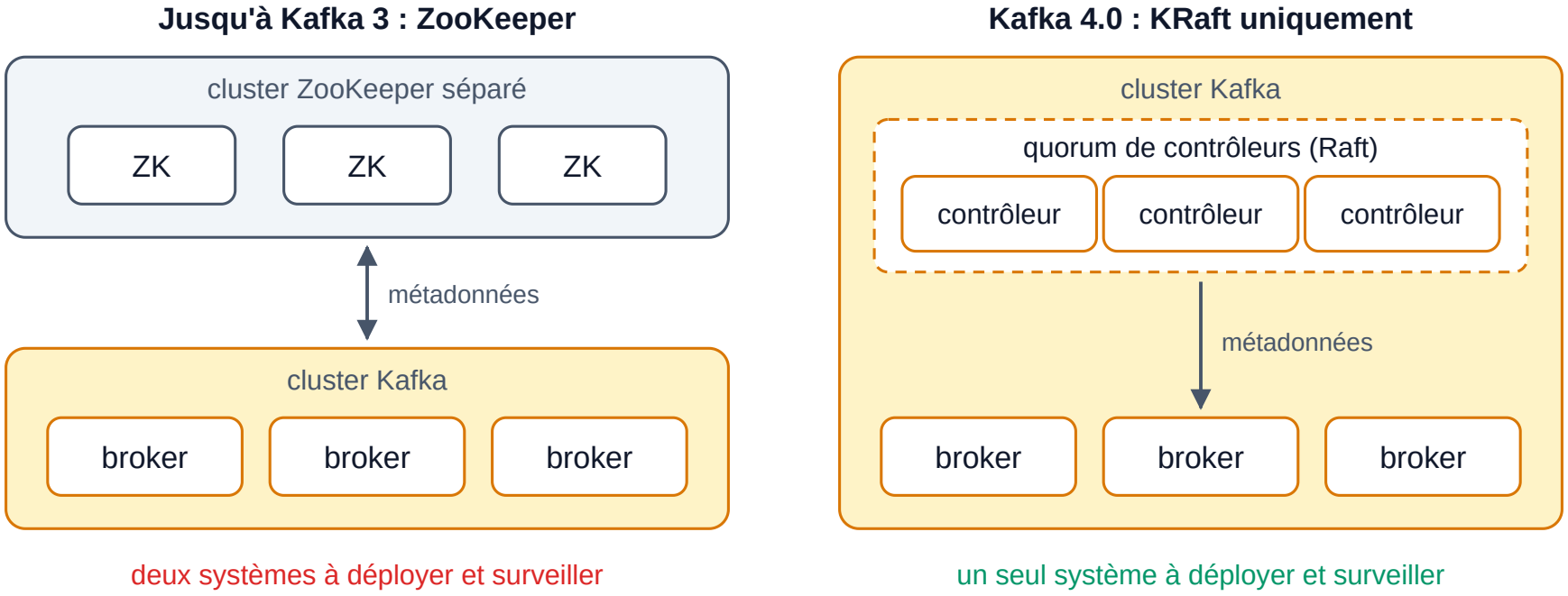


Schéma : avec KRaft, les métadonnées passent d'un cluster ZooKeeper externe à un quorum de contrôleurs intégré à Kafka.

4. Le temps et les fenêtres

4.1 Deux horloges

- Le **temps d'événement** : quand le paiement a eu lieu (`payment_ts`).
- Le **temps de traitement** : quand notre système le voit.

L'écart entre les deux est variable : réseau, application mobile hors ligne qui envoie ses événements plus tard, consommateur en retard. **Les événements arrivent dans le désordre.** Un calcul fondé sur le temps de traitement donne des résultats qui changent si l'on rejoue le flux ; un calcul fondé sur le temps d'événement donne le même résultat, mais il faut décider **combien de temps attendre** les retardataires.

4.2 Fenêtres

Fenêtre	Définition	Exemple ShopFlow
Fixe (*tumbling*)	Intervalles contigus de même durée	Nombre de paiements par minute et par pays
Glissante (*sliding / hopping*)	Durée fixe, démarre tous les pas	Nombre de cartes distinctes par IP sur 10 minutes, toutes les minutes
Session	Groupe d'événements séparés par une inactivité	Sessions de navigation (30 min d'inactivité)

4.2 Fenêtres (suite)

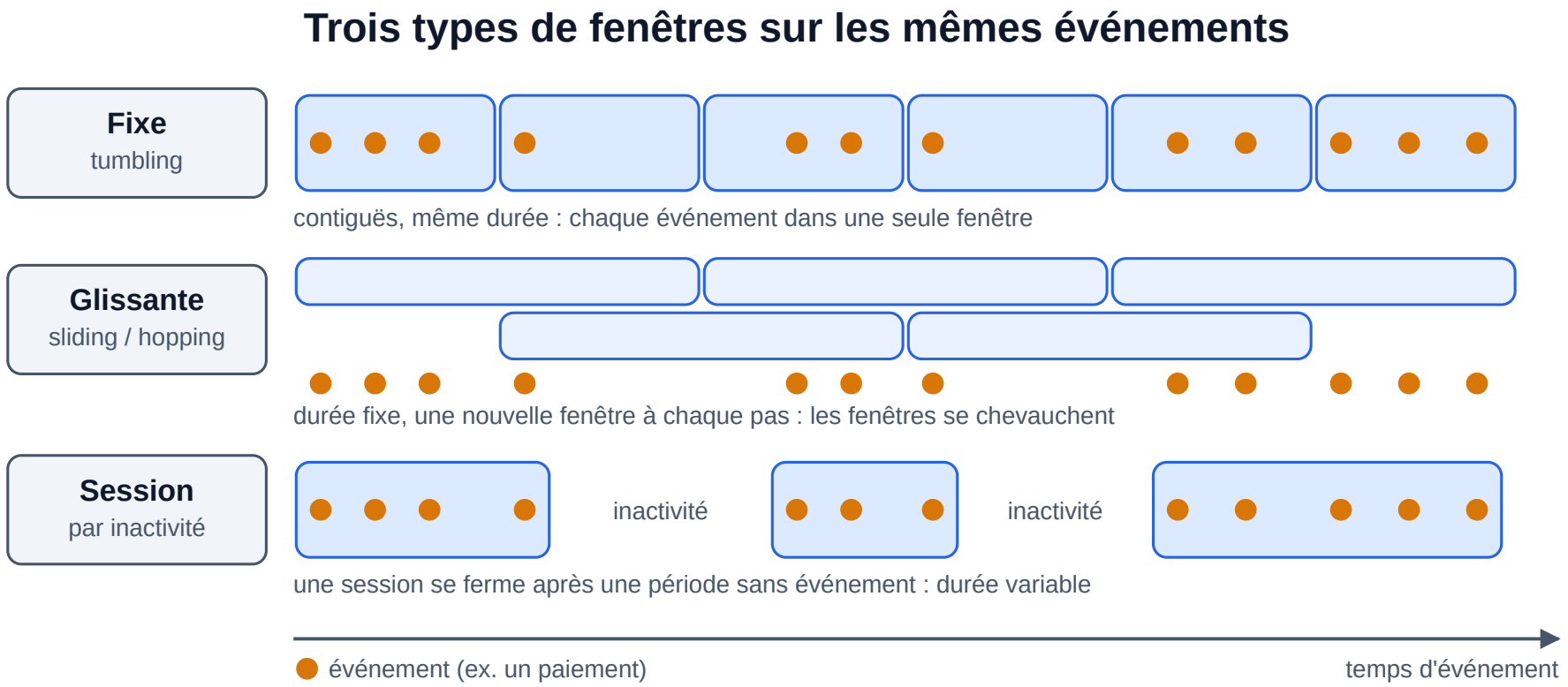


Schéma : un événement appartient à une seule fenêtre fixe, à plusieurs fenêtres glissantes, et les sessions ont une durée variable fixée par les silences (d'après Akidau et al., 2015).

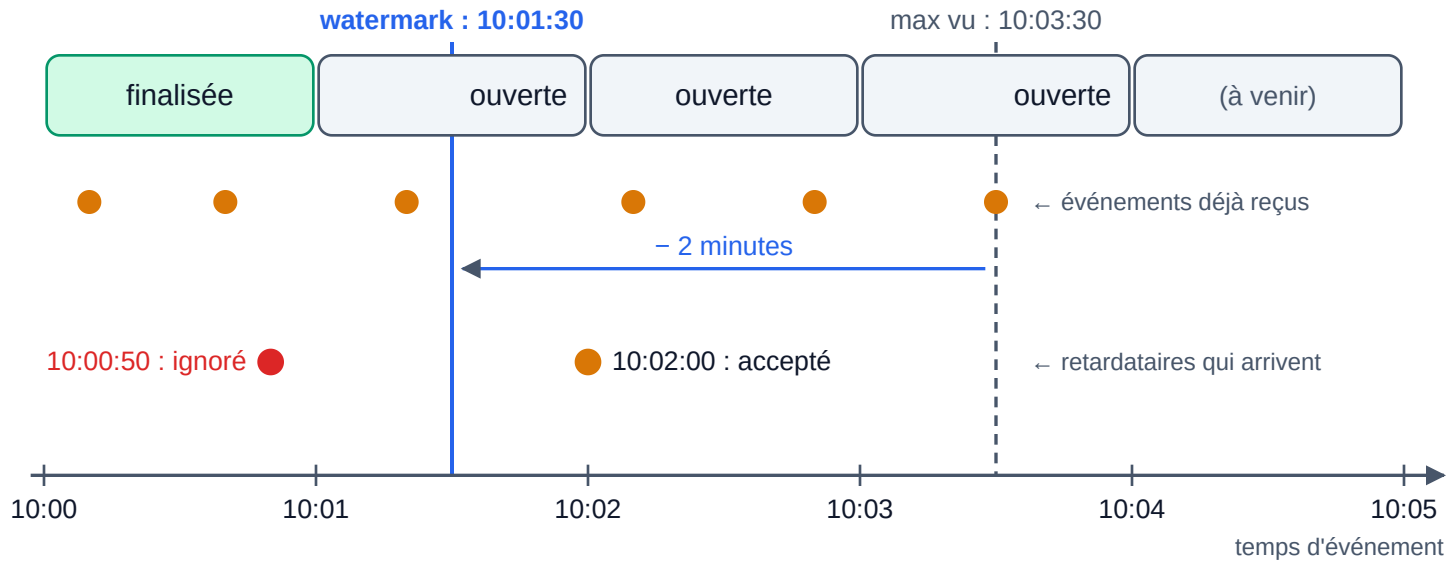
4.3 Watermarks

Un **watermark** est l'affirmation du système : « je ne m'attends plus à recevoir d'événement antérieur à T ». Dans Spark, `withWatermark("event_ts", "2 minutes")` fixe T = temps d'événement maximal vu − 2 minutes.

- Quand le watermark dépasse la fin d'une fenêtre, la fenêtre est **finalisée** : son résultat est émis (mode *append*) et son état est libéré.
- Un événement plus vieux que le watermark est **ignoré**.
- Compromis : un watermark large perd moins de retardataires mais retarde les résultats et garde plus d'état en mémoire.

4.3 Watermarks (suite)

Watermark de 2 minutes, fenêtres d'une minute



Le watermark a dépassé 10:01 : la fenêtre 10:00-10:01 est émise (append) et son état libéré.

Watermark plus large : moins de retardataires perdus, mais résultats plus tardifs et plus d'état.

Schéma : le watermark suit le temps d'événement maximal avec 2 minutes de retard ; la fenêtre qu'il a dépassée est finalisée, et un retardataire antérieur au watermark est ignoré.

5. Panorama des brokers et des moteurs

5.1 Brokers (stocker et distribuer le flux)

	Kafka	Redpanda	Apache Pulsar	Google Pub/Sub	Amazon Kinesis Data Streams
Modèle	Journal partitionné	Journal partitionné, API Kafka	Journal (BookKeeper) + brokers sans état	Service de messages managé	Journal partitionné (shards) managé
Déploiement	Auto-hébergé ou managé (Confluent, MSK, Managed Kafka de GCP)	Auto-hébergé ou cloud	Auto-hébergé ou cloud	Managé uniquement	Managé uniquement
Ordre	Par partition	Par partition	Par partition	Optionnel, par clé d'ordre	Par shard
Point fort	Écosystème, standard de fait	Pas de JVM, un binaire C++	Séparation calcul/stockage, multi-tenant	Zéro exploitation	Intégration AWS

On retrouve la séparation **calcul / stockage** de la séance 1 : Pulsar sépare les brokers (sans état) du stockage (BookKeeper)

5.2 Moteurs (calculer sur le flux)

	Spark Structured Streaming	Apache Flink	Kafka Streams	Apache Beam / Dataflow
Modèle d'exécution	Micro-batch (par défaut)	Événement par événement	Événement par événement, bibliothèque Java dans l'application	Modèle Dataflow, exécuté par un « runner » (Dataflow, Flink, Spark)
Latence typique	Secondes	Millisecondes	Millisecondes	Selon le runner
État et tolérance aux pannes	Checkpoints dans un stockage	Snapshots distribués asynchrones	Topics Kafka de changelog	Selon le runner
Point fort	Même API que le batch (DataFrame, SQL)	État volumineux, temps d'événement, faible latence	Pas de cluster à gérer	Portabilité, Dataflow managé

Au TP, nous utilisons **Spark Structured Streaming**. Pour la fraude en production, Flink est fréquent à cause de sa faible latence et de sa gestion d'état.

5.3 L'architecture ShopFlow à la fin de cette séance

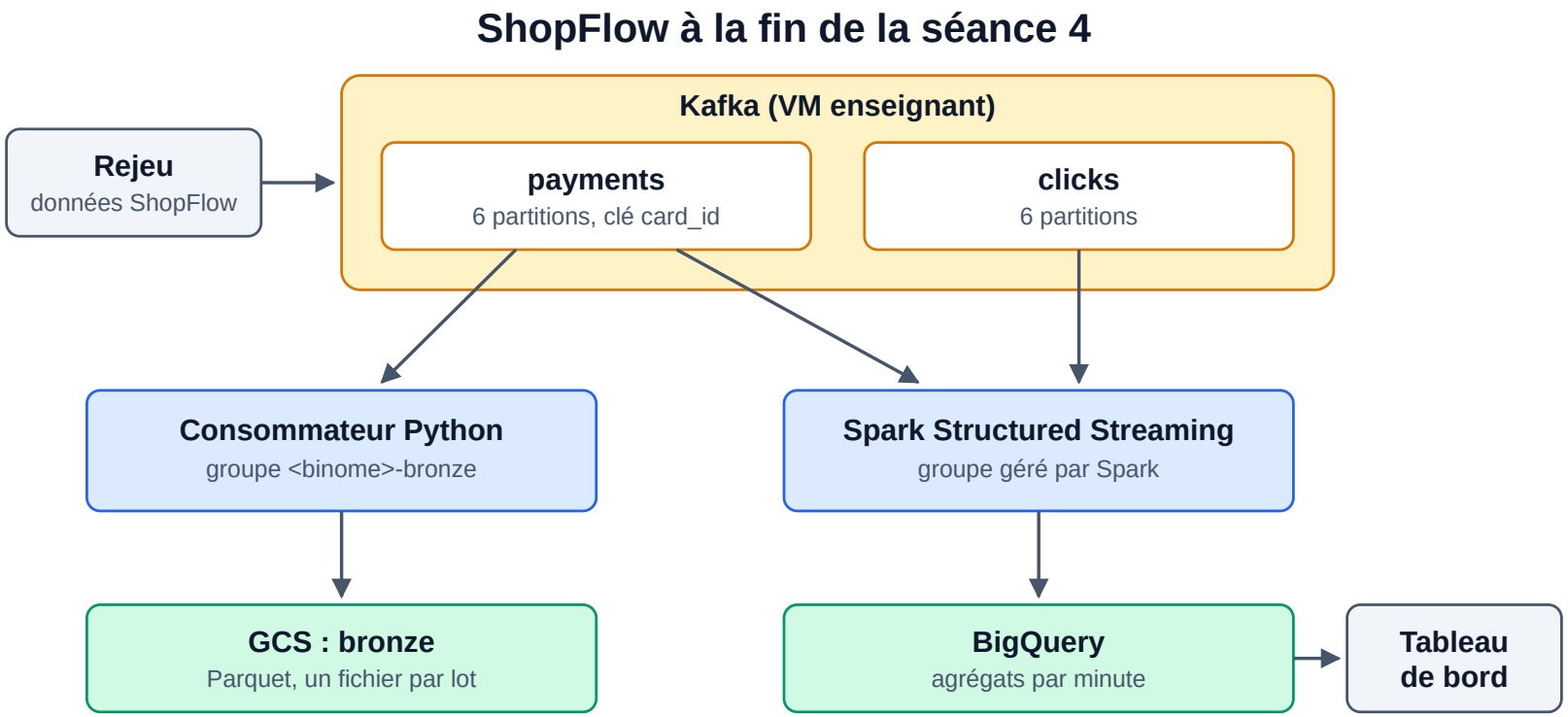


Schéma : le consommateur Python lit `payments` pour écrire le bronze sur GCS ; Spark lit `payments` et `clicks` pour produire les agrégats par minute dans BigQuery, affichés par le tableau de bord.